

NDNComm 2021

Developing practical E2E systems and applications using NDN

Marcin Spoczynski, Kathyayani Srikanteswara



10/28/2021

Abstract

Tight integration of Data, Networking, and Computational Power is one of the many benefits offered by the architecture of NDN for Edge and Cloud Deployments as opposed to IP-based Networked systems.

Further, multicasting, caching, or built-in security can provide many benefits to the operators and developers. We will explore how to build and deploy systems based on NDN along with the onboarding of practical applications. We will also uncover some of the gaps in the NDN stack for developing industrial IoT or cloud gaming applications and provide a recipe for future best practices for deployments of NDN networks and applications.

Outline

- What do we mean by practical system?
- Use cases
- Practical system components
- E2E System
- Applications
- Best practices
- Ideal system / Gaps

Practical system requirements

- Infrastructure
- Provisioning
- Runtime
- Orchestration and Management
- Application Definition & Development
- Monitoring, Logging & Auditing
- Culture

Ideal flexible E2E System

Simple	Each of the system components is responsible for one job
Adaptive	Automation can understand requirements for each host and applications
At scale	Components and agents can fit into different kinds of hardware and platforms
Predictable	If the system fails, I will know why and what action do I need to take

Use cases



Industrial IOT

Industrial Control Applications
Sensory Data
Control Plane



Gaming

WebRTC
Distributed rendering

Photos are licensed under [CC BY-SA-NC](#)

Practical System Components

Into Cloud native deployment

CI



Golang
Rust
Application Layer

Sensors

vPLC
openPLC



Orchestration Layer

Intel® RDT
DPDK
SPDK



Network Layer

Network Management System

YaNFD



KUBESEC.IO



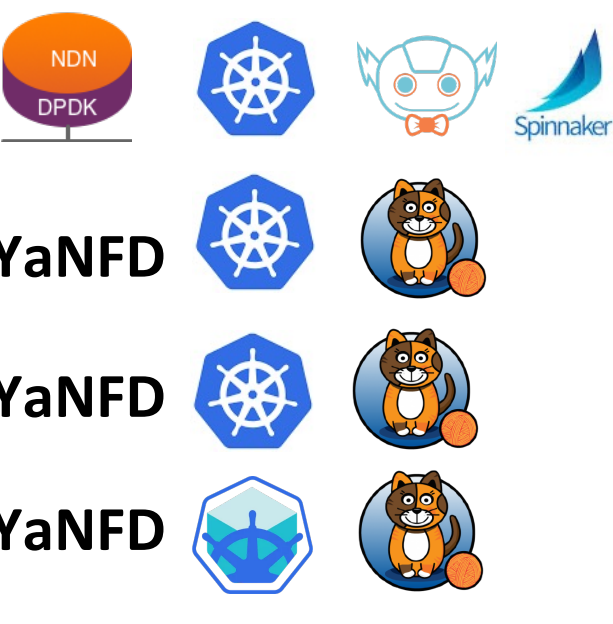
Observability / Telemetry

Stakeholders: Application Developers, System Builders

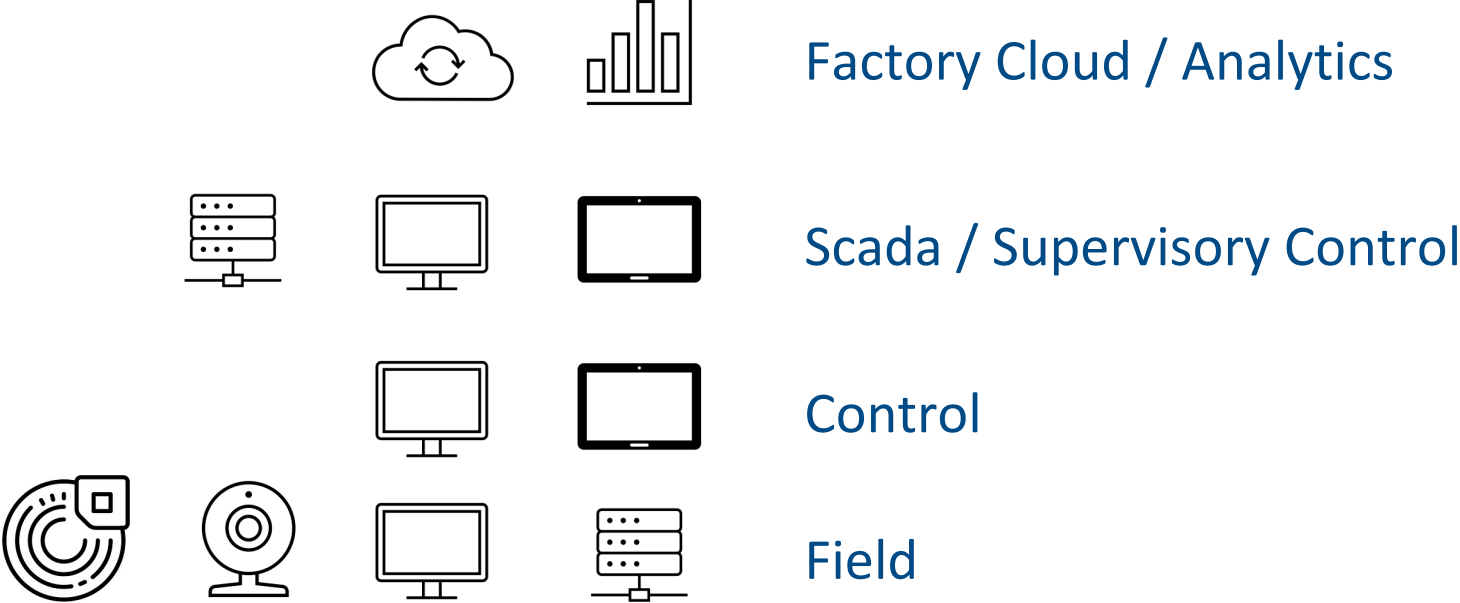
E2E System for Industrial Use Case

Enabling industrial Control on NDN

Software



Control Levels



Applications

Languages	90% of our code are made in Golang and Rust, integration and automation Python
Industrial Control	Example application, Virtual PLC based on the open PLC specification, Sensory data using OPC UA with NDN transport
Gaming	Pion WebRTC Golang framework with partial NDN transport
Integration	NFD and application controller, automated deployment through Kubernetes API

Application development / API

Application Logic	Application specific code, PLC, Video analytics etc.
Protocol Translation	Library that helps to translate protocols between many industrial standards e.g. MQTT, OPC UA etc.
Data Mapper	Helps mapping NDN address namespace to other protocols formats e.g., mDNS, consul etc..
Network Library	Network library NDN, TCP, Modbus etc..

Gaps

- Migration framework to port existing application to NDN
 - Translation and plug and play libraries for application developers e.g. Golang, Rust
 - Narrowed support for popular internet services and protocols e.g. WebRTC
- Lack of integration and automation scripts for most of the existing NDN forwarders e.g., Ansible, Chef
- Fully distributed control of NFD (caching, strategy) e.g., Distributed multiverse
- Time sensitive extension for NFD. e.g., TSN awareness

Conclusions

- NDN community made vital progress over last years
- NDN need to offer backwards portability to be widely adopted
- There is a need to develop E2E NDN based systems to showcase the feasibility
- Standardization of the end-user API interfaces required to provide stack and defragment software
- Offloading for Storage, Network, and Compute is needed to be competitive with other network paradigms
- Provide easy way to port existing application to NDN.

